MongoDB Commands by Use Case

MovieMatch — quick reference. For connecting and setup, see *MovieMatch-MongoDB-Guide.pdf*.

> **Read this once, then skip to your use case.** Each interactor depends on its own data-access *interface* in `use_case/`; the single `MongoUserDataAccessObject` in `data_access/` implements them all. Only that one class ever imports `com.mongodb`, and only one `MongoClient` exists for the whole app — build it once in `AppBuilder` and pass the same instance to every interactor.

Every command at a glance

Use case	Command
Signup — name taken?	`find(...).first() != null`
Signup — store user	`insertOne(doc)`
Login — fetch user	`find(...).first()`
Logout	*no database call*
Change password	`updateOne(..., Updates.set)`
Security question — read Q&A	`find(...).first()`
Reset password	`updateOne(..., Updates.set)`
Delete account	`deleteOne(...)`
Edit profile	`updateOne(..., Updates.set)`
Watchlist — add	`updateOne(..., Updates.addToSet)`
Watchlist — remove	`updateOne(..., Updates.pull)`
Watchlist — read	`find(...)` + `getList`
Mark watched + rate	`updateOne(..., push + pull)`
Recommendations — friends' ratings	`find(Filters.in(...))`

Every command needs these two imports:

```
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Updates;
```

The commands, use case by use case

Signup

```
// is the username taken?
boolean taken = users.find(Filters.eq("username", username)).first() != null;

// store the new user
users.insertOne(new Document("username", username)
        .append("displayName", displayName)
        .append("password", password)
        .append("securityQuestion", question)
        .append("answer", answer)
        .append("watchlist", new ArrayList<String>())
        .append("watched", new ArrayList<Document>()));
```

Login

```
Document doc = users.find(Filters.eq("username", username)).first();
// doc == null  -> no such account
// then compare doc.getString("password") with what was typed
```

Logout

No database call. The current user is session state — keep it in a field on the DAO.

```
private String currentUsername;   // set on login, cleared on logout
```

Change password / Reset password

```
users.updateOne(Filters.eq("username", username),
                Updates.set("password", newPassword));
```

Security question

```
Document doc = users.find(Filters.eq("username", username)).first();
String question = doc.getString("securityQuestion");
String answer   = doc.getString("answer");
```

Delete account

```
long removed = users.deleteOne(Filters.eq("username", username)).getDeletedCount();
// removed == 1 -> the account was deleted
```

Edit profile

```
users.updateOne(Filters.eq("username", username),
                Updates.set("displayName", displayName));
```

Watchlist

Stored as an array inside the user document. `addToSet` instead of `push` so the same title can't be saved twice.

```
// add
users.updateOne(Filters.eq("username", username), Updates.addToSet("watchlist", mediaId));

// remove
users.updateOne(Filters.eq("username", username), Updates.pull("watchlist", mediaId));

// read
Document doc = users.find(Filters.eq("username", username)).first();
List<String> watchlist = doc.getList("watchlist", String.class);
```

Mark watched + rate

One update does both: record the rating and drop it from the watchlist.

```
Document entry = new Document("mediaId", mediaId).append("rating", rating);
users.updateOne(Filters.eq("username", username),
                Updates.combine(Updates.push("watched", entry),
                                Updates.pull("watchlist", mediaId)));

// read back
List<Document> watched = doc.getList("watched", Document.class);
```

Recommendations

```
// titles this user rated 4+, for the taste profile
for (Document entry : doc.getList("watched", Document.class)) {
    if (entry.getInteger("rating") >= 4) {
        profile.add(entry.getString("mediaId"));
    }
}

// how friends rated one title — one query for all friends
for (Document friend : users.find(Filters.in("username", friendUsernames))) {
    // scan friend.getList("watched", Document.class) for mediaId
}
```

Document shape

```
{
  "username": "bob",
  "displayName": "Bob",
  "password": "oldpass",
  "securityQuestion": "What is your pet's name?",
  "answer": "Fido",
  "watchlist": ["tt0111161", "tt0068646"],
  "watched": [ { "mediaId": "tt1375666", "rating": 5 } ]
}
```

Field names are case-sensitive. A typo returns `null` rather than an error.

Adding a use case

Declare what you need, then let the DAO implement it. Delete-account as the example:

```
package use_case.delete_account;

public interface DeleteAccountUserDataAccessInterface {
    boolean existsByName(String username);
    boolean deleteUser(String username);
}
```

Add that interface to `MongoUserDataAccessObject implements ...` and write the method there. Your interactor never sees MongoDB.

The DAO

Compiles against the team's existing interfaces.

```
package data_access;

import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;

import org.bson.Document;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Updates;

import entity.StandardUser;
import entity.User;
import use_case.change_password.ChangePasswordUserDataAccessInterface;
import use_case.login.LoginUserDataAccessInterface;
import use_case.logout.LogoutUserDataAccessInterface;
import use_case.reset_password.ResetPasswordUserDataAccessInterface;
import use_case.security_question.SecurityQuestionUserDataAccessInterface;
import use_case.signup.SignupUserDataAccessInterface;

public class MongoUserDataAccessObject implements
        SignupUserDataAccessInterface,
        LoginUserDataAccessInterface,
        LogoutUserDataAccessInterface,
        ChangePasswordUserDataAccessInterface,
        SecurityQuestionUserDataAccessInterface,
        ResetPasswordUserDataAccessInterface {

    private static final String USERNAME = "username";
    private static final String DISPLAY_NAME = "displayName";
    private static final String PASSWORD = "password";
    private static final String SECURITY_QUESTION = "securityQuestion";
    private static final String ANSWER = "answer";
    private static final String WATCHLIST = "watchlist";
    private static final String WATCHED = "watched";

    private final MongoClient mongoClient;
    private final MongoCollection<Document> users;
    private String currentUsername;
```

```
public MongoUserDataAccessObject(String propertiesPath) {
    final Properties props = new Properties();
    try (InputStream in = new FileInputStream(propertiesPath)) {
        props.load(in);
    }
    catch (IOException ex) {
        throw new RuntimeException("Could not read " + propertiesPath, ex);
    }
    this.mongoClient = MongoClients.create(props.getProperty("uri"));
    final MongoDatabase database = mongoClient.getDatabase(props.getProperty("database"));
    this.users = database.getCollection(props.getProperty("collection"));
}

@Override
public boolean existsByName(String username) {
    return users.find(Filters.eq(USERNAME, username)).first() != null;
}

@Override
public void save(User user) {
    if (existsByName(user.getUsername())) {
        users.updateOne(Filters.eq(USERNAME, user.getUsername()),
                Updates.combine(
                        Updates.set(DISPLAY_NAME, user.getDisplayName()),
                        Updates.set(PASSWORD, user.getPassword()),
                        Updates.set(SECURITY_QUESTION, user.getSecurityQuestion()),
                        Updates.set(ANSWER, user.getSecurityAnswer())));
    }
    else {
        users.insertOne(new Document(USERNAME, user.getUsername())
                .append(DISPLAY_NAME, user.getDisplayName())
                .append(PASSWORD, user.getPassword())
                .append(SECURITY_QUESTION, user.getSecurityQuestion())
                .append(ANSWER, user.getSecurityAnswer())
                .append(WATCHLIST, new ArrayList<String>())
                .append(WATCHED, new ArrayList<Document>()));
    }
}

@Override
public User get(String username) {
    final Document doc = users.find(Filters.eq(USERNAME, username)).first();
    if (doc == null) {
        return null;
    }
    return toUser(doc);
```

```
    }

    @Override
    public String getCurrentUsername() {
        return currentUsername;
    }

    @Override
    public void setCurrentUsername(String username) {
        this.currentUsername = username;
    }

    @Override
    public void changePassword(User user) {
        users.updateOne(Filters.eq(USERNAME, user.getUsername()),
                Updates.set(PASSWORD, user.getPassword()));
    }

    @Override
    public void changePassword(String username, String newPassword) {
        users.updateOne(Filters.eq(USERNAME, username), Updates.set(PASSWORD, newPassword));
    }

    public boolean deleteUser(String username) {
        return users.deleteOne(Filters.eq(USERNAME, username)).getDeletedCount() > 0;
    }

    public void changeDisplayName(String username, String displayName) {
        users.updateOne(Filters.eq(USERNAME, username), Updates.set(DISPLAY_NAME, displayName));
    }

    public void addToWatchlist(String username, String mediaId) {
        users.updateOne(Filters.eq(USERNAME, username), Updates.addToSet(WATCHLIST, mediaId));
    }

    public void removeFromWatchlist(String username, String mediaId) {
        users.updateOne(Filters.eq(USERNAME, username), Updates.pull(WATCHLIST, mediaId));
    }

    public List<String> getWatchlist(String username) {
        final Document doc = users.find(Filters.eq(USERNAME, username)).first();
        if (doc == null || doc.getList(WATCHLIST, String.class) == null) {
            return new ArrayList<>();
        }
        return doc.getList(WATCHLIST, String.class);
    }
```

```
    public void markWatched(String username, String mediaId, int rating) {
        final Document entry = new Document("mediaId", mediaId).append("rating", rating);
        users.updateOne(Filters.eq(USERNAME, username),
                Updates.combine(
                        Updates.push(WATCHED, entry),
                        Updates.pull(WATCHLIST, mediaId)));
    }

    public List<Document> getWatched(String username) {
        final Document doc = users.find(Filters.eq(USERNAME, username)).first();
        if (doc == null || doc.getList(WATCHED, Document.class) == null) {
            return new ArrayList<>();
        }
        return doc.getList(WATCHED, Document.class);
    }

    public List<String> getHighlyRatedMedia(String username, int minRating) {
        final List<String> result = new ArrayList<>();
        for (Document entry : getWatched(username)) {
            final Integer rating = entry.getInteger("rating");
            if (rating != null && rating >= minRating) {
                result.add(entry.getString("mediaId"));
            }
        }
        return result;
    }

    public List<Integer> getFriendRatings(List<String> friendUsernames, String mediaId) {
        final List<Integer> ratings = new ArrayList<>();
        for (Document doc : users.find(Filters.in(USERNAME, friendUsernames))) {
            final List<Document> watched = doc.getList(WATCHED, Document.class);
            if (watched == null) {
                continue;
            }
            for (Document entry : watched) {
                if (mediaId.equals(entry.getString("mediaId")) && entry.getInteger("rating") != null) {
                    ratings.add(entry.getInteger("rating"));
                }
            }
        }
        return ratings;
    }

    private User toUser(Document doc) {
        return new StandardUser(
                doc.getString(USERNAME),
                doc.getString(DISPLAY_NAME),
```

```
                doc.getString(PASSWORD),
                doc.getString(SECURITY_QUESTION),
                doc.getString(ANSWER));
    }

    public void close() {
        mongoClient.close();
    }
}
```